

Université Hassan II

Ecole Normale supérieure de l'Enseignement et Technique

Département Mathématiques et Informatique

Projet gestion de projet

Rapport controle JEE

Réalisés par :

BEN TALEB Ilyasse

Encadrée par :

**Pr. Mohamed
YOUSSEFI**

Année universitaire **2025–2026**

11 mai 2026

Table des matières

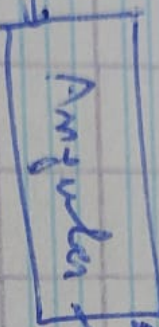
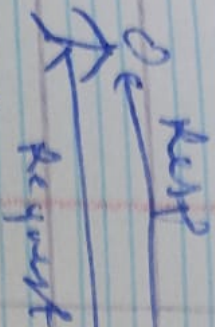
1	Architecture Technique	2
2	Conception : Diagramme de Classes	4
3	Implémentation Backend	4
3.1	Couche DAO	4
3.2	Couche Service & DTOs	8
3.3	Controller	9
3.4	Frontend	10
4	Sécurité JWT et Rôles	10

1 Architecture Technique

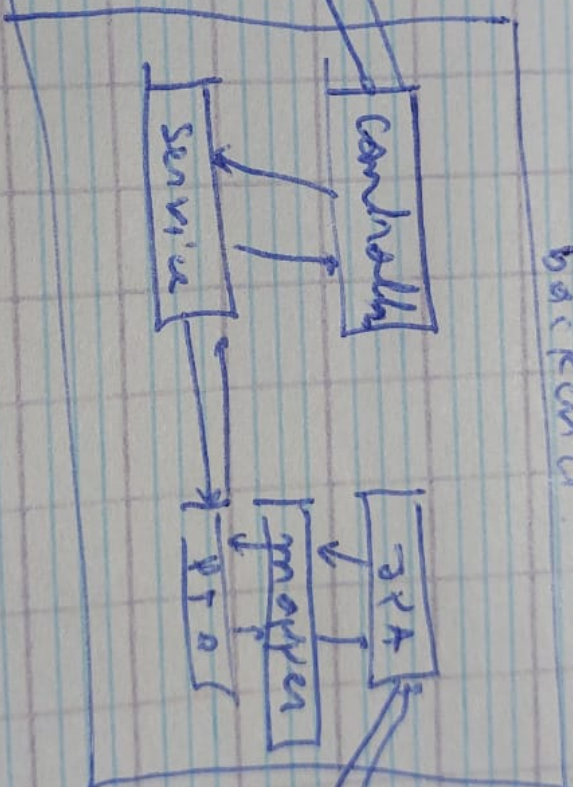
L'application adopte une architecture simulée par un backend monolithique modulaire :

- **Frontend** : Angular .
- **Backend** : Spring Boot 3.2.
- **Sécurité** : JWT (Stateless).

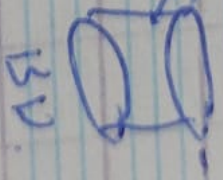
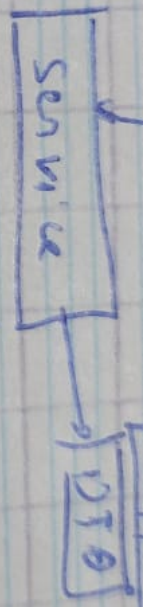
Client



frontend



backend



2 Conception : Diagramme de Classes

Le digramme de classe de le projet

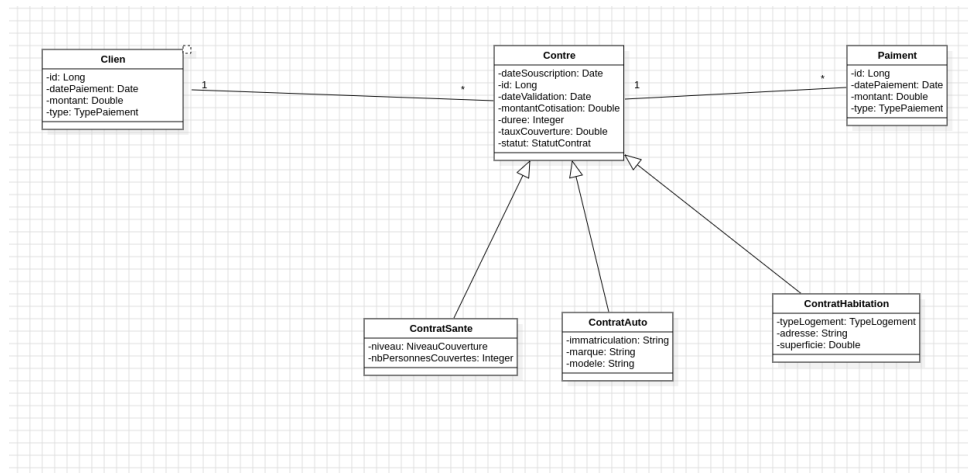


FIGURE 2 – Diagramme de classes des entités JPA

3 Implémentation Backend

3.1 Couche DAO

Voici les entites de projet

```

package net.ilyasse.bentaleb.backend.entity;

import jakarta.persistence.*;
import lombok.Data;
import lombok.Getter;
import lombok.Setter;

import java.util.List;

@Entity
@Getter
@Setter
@Data
public class Client {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    @OneToMany(mappedBy = "client")
    private List<Contra> contras;
}

```

FIGURE 3 – Client Entit

```

@Entity
3 inheritors
@Getter
@Setter
@Data
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Contra {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Date dateSouscription;
    private StatutContra statut ;
    private Date dateValidation;
    private double cotisation;
    private int dure;
    private double tauxCouverture;
    @ManyToOne
    private Client client;
    @OneToMany(mappedBy = "contra")
    private List<Paiement> paiements;
}

```

FIGURE 4 – ContraEntit

```

package net.ilyasse.bentaleb.backend.entity;

import jakarta.persistence.Entity;
import lombok.Data;

@Entity
@Data
public class ContratAuto extends Contra{
    private String immatriculation;
    private String marque;
    private String modele;
}

```

FIGURE 5 – Contra de type automobile

```

package net.ilyasse.bentaleb.backend.entity;

import jakarta.persistence.Entity;
import lombok.Data;

@Entity
@Data
public class ContratSante extends Contra{
    private Integer nbPersonnesCouvertes;
    private NiveauCouverture niveau;
}

```

FIGURE 6 – Contra de type sante

```

import java.util.Date;

@Entity
@Getter
@Setter
@Data
public class Paiment {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Date date;
    private double montant;
    private TypePaiment typePaiment;
    @ManyToOne
    private Contra contra;
}

```

FIGURE 7 – paiment entite

voici les classe enum qui j'ai utilisé en ce projet

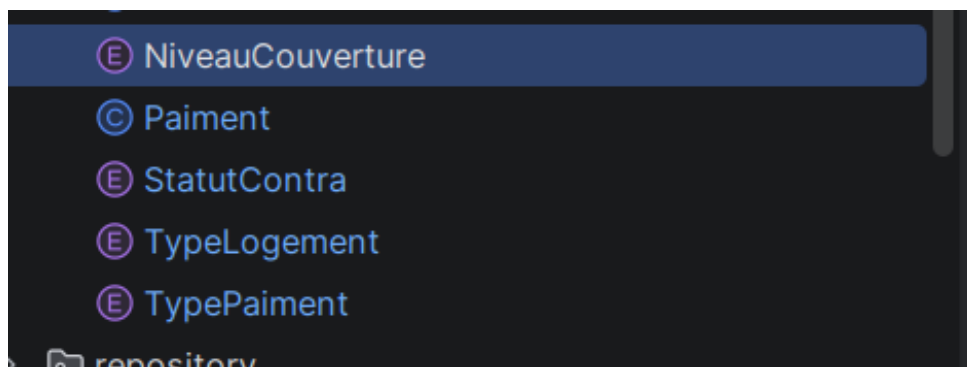


FIGURE 8 – Les classe Enume

maintenant voici l'aichitecture globale de jpa

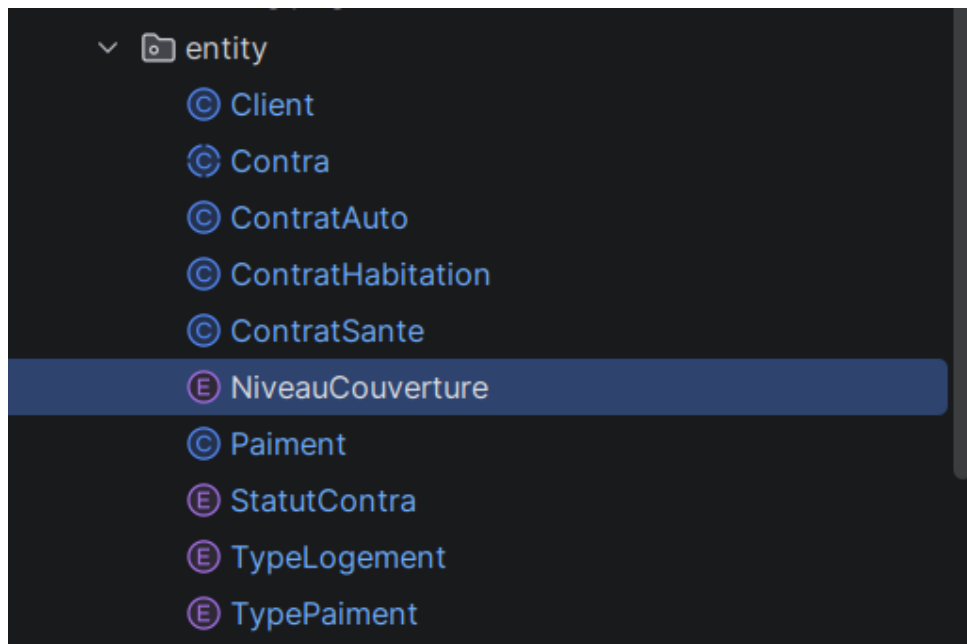


FIGURE 9 – Architecture jpa

3.2 Couche Service & DTOs

Les DTOs de tout les entites

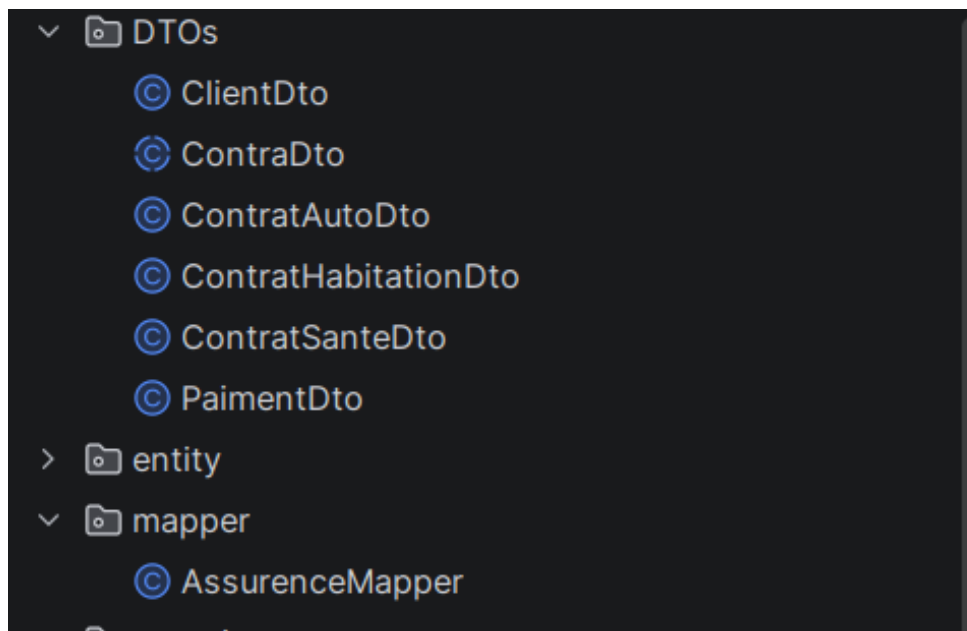


FIGURE 10 – les classes DTO

et les prncipe fonctionnalité que j'ai réalisé

```

package net.ilyasse.bentaleb.backend.service;

import net.ilyasse.bentaleb.backend.DTOs.ClientDto;
import net.ilyasse.bentaleb.backend.DTOs.ContraDto;

import java.util.List;

public interface AssuranceService { no usages new *
    ClientDto ajouteClient(ClientDto clientDto); no usages new *
    void deleteClient(ClientDto clientDto); no usages new *
    List<ClientDto> getAllClient(ClientDto clientDto); no usages new *
    ContraDto ajouteContra(ContraDto contraDto); no usages new *
    List<ContraDto> getContraUser(Long clientId); no usages new *
}

```

FIGURE 11 – l'interface service

3.3 Controller

j'ai réalisé deux controller controller client

```

@RestController
@CrossOrigin("*")
@RequestMapping("/api/client")
public class ClientController {
    @Autowired
    private AssuranceServiceImp assuranceServiceImp;
    @GetMapping()
    public List<ClientDto> getAllclient(){
        return assuranceServiceImp.getAllClient();
    }
    @GetMapping("/{clientId}")
    public ClientDto getClient(@PathVariable Long clientId){
        return assuranceServiceImp.getClient(clientId);
    }
    @PostMapping()
    public ClientDto addClient(@RequestBody ClientDto dto){
        return assuranceServiceImp.ajouteClient(dto);
    }
    @DeleteMapping("/{clientId}")
    public void deleteClient(@PathVariable Long clientId){
        assuranceServiceImp.deleteClient(clientId);
    }
}

```

FIGURE 12 – le controller client

et controller contrat

```

import java.util.List;

@RestController  @ ok
@CrossOrigin("*")
@RequestMapping("/api/contrats")
public class ContraController {
    @Autowired
    private AssuranceDerviceImp assuranceDerviceImp;
    @GetMapping("/{clientId}")  @ ok
    public List<ContraDto> getAllclientContra(@PathVariable long clientId){
        return assuranceDerviceImp.getContraUser(clientId);
    }
    @GetMapping("/{clientId}")  @ ok
    public ClientDto getClient(@PathVariable Long clientId){
        return assuranceDerviceImp.getClient(clientId);
    }
    @PostMapping()  @ ok
    public ContraDto addConte(@RequestBody ContraDto dto){
        return assuranceDerviceImp.ajouteContra(dto);
    }
}

```

FIGURE 13 – le controller contrat

3.4 Frontend

j'ai réaliser un mvp de client en frontend

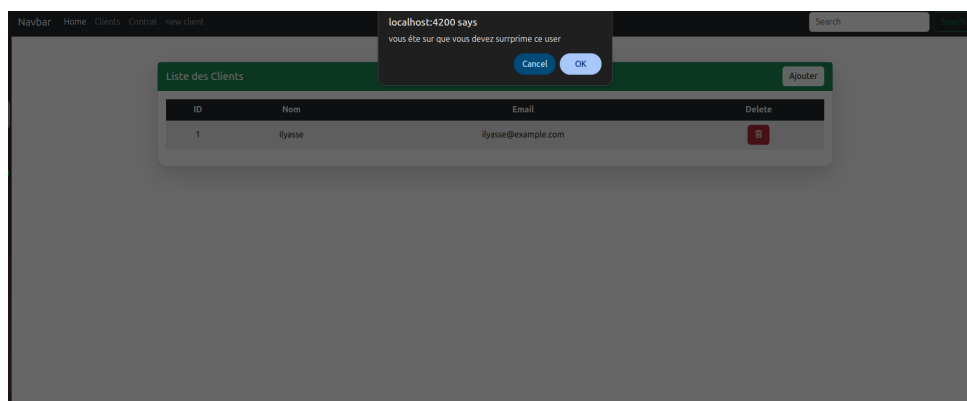


FIGURE 14 – lFrontend

4 Sécurité JWT et Rôles